

从 Softmax TD 评估 到两阶段 Q 控制

给科研初学者的难点、推导与边界

本文只回答一个问题：

为什么这样做

前人已经证明 softmax attention 能做 TD policy evaluation，为什么把它推进到 Q-control 仍然困难？我们的两阶段方法又究竟解决了什么？

核心答案是：难点不是 TD error 的正负号，而是 Bellman optimality 所需的动作比较，以及更新结果能否写回正确的 state-action query。

阅读前的符号与假设

- m 或 $|A|$: 有限动作集合的大小; $\log$ 表示自然对数。
- α : 更新步长, 理论部分假设 $0 < \alpha \leq 1$ 。
- γ : 折扣因子, 假设 $0 \leq \gamma < 1$; reward 有界。
- β : Stage I 的动作 softmax 锐度, $\beta > 0$ 。
- η : Stage II 的 state-action kernel 锐度, $\eta > 0$ 。
- T-star: Bellman optimality operator;
T-beta: 本文的近似算子。
- ϵ_K : kernel 相对 identity write-back 的偏差。
- ϵ_n : empirical target 相对 conditional expectation 的统一误差上界。
- B : 所有相关 signed TD values 的绝对值统一上界。

“operator-level construction” 表示: 我们证明固定代数模块能够执行所写更新; 它

不等于已经证明普通 pretrainedTransformer 会通过训练自动学出这些模块。

为什么这样做

科研推导首先要写清楚“对象、假设、结论”。同一个公式脱离假设后，可能不再成立。

1. 先把研究问题说准确

我们要从固定策略评价推进到控制。固定策略评价只需要执行一个已经给定的 Bellman expectation; Q-learning 路线则必须在下一状态的动作之间进行 greedification。

两条算子分别是：

$$(T^{\pi}Q)(s, a) = \mathbb{E}\left[R + \gamma \sum_b \pi(b | S') Q(S', b) \mid s, a\right]$$

$$(T^*Q)(s, a) = \mathbb{E}\left[R + \gamma \max_b Q(S', b) \mid s, a\right]$$

二者真正新增的操作只有一处，但非常关键：

$$\sum_b \pi(b | s') Q(s', b) \rightarrow \max_b Q(s', b)$$

最容易误解

“策略改进一定要把 max 写进每一个 TD residual” 不是普遍命题。actor-critic、SARSA 等路线也能改进策略。max 是我们选择 Q-learning / Bellman optimality 路线之后必须处理的操作。

2. 三项工作的边界

Xie 路线

Xie 等人的 softmax TD evaluation 已经提供：

$$\Delta v(s) = \sum_k K(s_k, s) \delta_k$$

其中匹配核 K 为正，但 TD error 可以为正或负。这说明 softmax 的正权重并不会自动删除 value 中的负号。

Liang--Lai 路线

Liang--Lai 研究参数化 action value：

$$Q_w(s, a) = \phi(s, a)^\top w$$

其 semi-gradient update 需要动态形成每个样本的乘积 $\delta \times \phi$ 。linear attention 的二阶矩结构适合完成这一乘法。

本文路线

我们不更新全局参数 w ，而是直接把每个 $Q(s,a)$ 存在 query token 上。这样把问题改写成：

- 1 用 softmax 在同一下一状态的动作中近似 max;
- 2 用 Xie 型 softmax kernel 把 signed TD value 写回正确 query。

答辩表述

本文不是 Liang--Lai 参数化结果的无条件严格超集。我们用更直接的 tabular token memory 换取了标准 softmax 构造。

3. 为什么正权重不是符号障碍

一个 softmax head 的标量输出可以写成：

$$y = \sum_k a_k v_k, \quad a_k \geq 0, \quad \sum_k a_k = 1$$

若 value 是：

$$(v_1, v_2) = (-2, 1)$$

权重是：

$$(a_1, a_2) = (0.8, 0.2)$$

则输出为：

$$y = 0.8(-2) + 0.2(1) = -1.4$$

因此“权重为正”和“输出为正”是两个不同命题。softmax 只约束怎样平均，不约束被平均的 value 必须非负。

最容易误解

凸包结论是 y 位于 value 集合的凸包中。如果 value 本身含有负的 TD error，凸包当然也可以包含负数。

4. 凸组合真正限制的是 max

令一个状态下的动作值为：

$$q = (q_1, \dots, q_m)$$

定义 softmax 权重及其加权值：

$$p_\beta(a; q) = \frac{e^{\beta q_a}}{\sum_b e^{\beta q_b}}$$

$$M_\beta(q) = \sum_a p_\beta(a; q) q_a$$

因为 p 是概率向量， M_β 是 q 的凸组合，所以：

$$\min_a q_a \leq M_\beta(q) \leq \max_a q_a$$

当 β 有限且至少有一个动作严格次优时，所有动作权重仍严格为正，因此：

$$M_\beta(q) < \max_a q_a$$

这就是 convexity 对我们 Q-learning 路线的真实影响：它造成有限温度下的 max 低估，而不是造成 TD 符号消失。

5. 一个可核验的 max 近似例子

令下一状态的三个动作值为：

$$q = (1, 2, 4), \quad \beta = 2$$

softmax 权重约为：

$$p_2 = (0.00243, 0.01794, 0.97963)$$

所以：

$$M_2(q) = 0.00243(1) + 0.01794(2) + 0.97963(4)$$

$$M_2(q) \approx 3.9569 < 4 = \max_a q_a$$

实际 gap 约为 0.0431。后面将证明统一上界：

$$\max_a q_a - M_\beta(q) \leq \frac{\log m}{\beta}$$

在本例中右侧为：

$$\frac{\log 3}{2} \approx 0.5493$$

实际 gap 小于理论上界。上界不一定紧，但它对任意 q 都成立。

6. Stage I: grouped softmax greedification

对 transition k :

$$(s_k, a_k, r_k, s'_k)$$

上下文必须提供属于下一状态 s'_k 的全部动作 token:

$$Q(s'_k, 1), \dots, Q(s'_k, m)$$

Stage I 只在这一组 token 内计算:

$$p_k(b) = \text{softmax}_b(\beta Q(s'_k, b))$$

$$M_\beta Q(s'_k) = \sum_b p_k(b) Q(s'_k, b)$$

group mask 的职责是阻止 s'_k 的查询错误关注其他状态的动作 token。然后形成近似 optimality TD error:

$$\delta_k^\beta = r_k + \gamma M_\beta Q(s'_k) - Q(s_k, a_k)$$

操作账本为什么重要

Stage I 的动作聚合没有调用 `exact max`，而是使用 `softmax` 加权求和；`exact max` 只存在于 **OracleMaxKernelQTD** 对照中。当前 `operator-level` 代码仍使用 `tensor indexing` 组织动作组，并包含线性组合与 `residual update`。Stage I 入口是 **`model.py::GroupedSoftmaxMax`**。

需要逐项核对：动作聚合由 `grouped softmax` 完成，`signed TD value` 由 `reward`、Stage-I 输出与当前 `Q` 线性组合得到，`state-action credit assignment` 由第二阶段 `kernel` 完成。

为什么这样做

我们必须显式列操作账本，否则很容易在 `attention` 外部先计算 `max`，再误称 `Transformer` 内部实现了 `Q-learning target`。

7. softmax-max 熵界：第一步

记：

$$L_{\beta}(q) = \frac{1}{\beta} \log \sum_a e^{\beta q_a}$$

从 softmax 定义可得：

$$\log p_a = \beta q_a - \log \sum_b e^{\beta q_b}$$

两侧乘以 p_a 并对 a 求和：

$$\sum_a p_a \log p_a = \beta \sum_a p_a q_a - \log \sum_b e^{\beta q_b}$$

利用：

$$H(p) = -\sum_a p_a \log p_a$$

以及 M_{β} 的定义，整理得到 Gibbs entropy identity：

$$L_{\beta}(q) = M_{\beta}(q) + \frac{H(p)}{\beta}$$

这一步非常重要：log-sum-exp 与 softmax 加权平均不是同一个量，它们相差“熵除以温度锐度”。

8. softmax-max 熵界：第二步

令：

$$q_{\max} = \max_a q_a$$

因为指数和中至少包含最大动作对应的那一项，所以：

$$L_{\beta}(q) \geq q_{\max}$$

另一方面， M_{β} 是动作值的凸组合，因此：

$$M_{\beta}(q) \leq q_{\max}$$

结合上一页的 entropy identity：

$$q_{\max} - M_{\beta}(q) \leq L_{\beta}(q) - M_{\beta}(q)$$

$$q_{\max} - M_{\beta}(q) \leq \frac{H(p)}{\beta}$$

包含 m 个动作的概率分布具有熵上界，因此：

$$0 \leq q_{\max} - M_{\beta}(q) \leq \frac{\log m}{\beta}$$

为什么这样做

这个界使 “softmax 可以近似 max” 从直觉变成了可进入 Bellman 误差分析的统一量结论。

9. 从 max gap 到 Bellman target gap

定义 approximate Bellman operator:

$$(T_\beta Q)(s, a) = \mathbb{E}[R + \gamma M_\beta(Q(S', \cdot)) \mid s, a]$$

真正的 optimality operator 是:

$$(T^* Q)(s, a) = \mathbb{E}[R + \gamma \max_b Q(S', b) \mid s, a]$$

对每个可能的下一状态, Stage I gap 都受上一节的统一上界控制。取条件期望不会放大这个上界, 再乘折扣因子 gamma, 得到:

$$\|T_\beta Q - T^* Q\|_\infty \leq \gamma \frac{\log |\mathcal{A}|}{\beta}$$

这说明 beta 控制的是 Bellman target 偏差, 而不只是 attention 看起来是否尖锐。

答辩表述

固定有限 beta 时, 我们声称的是 approximate Bellman optimality, 而不是 exact max 或 exact Q-learning。

10. Stage II: signed kernel TD write-back

Stage I 已经产生 signed scalar:

$$\delta_k^\beta = r_k + \gamma M_\beta Q(s'_k) - Q(s_k, a_k)$$

令 state-action pair 为 $x_k=(s_k, a_k)$, 查询为 $x=(s, a)$ 。Stage II 的匹配分数是:

$$u_{k,x} = \eta \langle \psi(x_k), \psi(x) \rangle$$

在 context 维进行 softmax:

$$K_\eta(x_k, x) = \text{softmax}_k(u_{k,x})$$

把 delta 放在 value 中, 得到:

$$Q^+(x) = Q(x) + \alpha \sum_k K_\eta(x_k, x) \delta_k^\beta$$

K 回答 “更新哪一个 query” , delta 回答 “向上还是向下更新” 。

代码对应: **model.py::KernelizedSoftmaxQTD**。

11. Stage II 为什么不怕负号

假设某个 query 对两条经验的 kernel 权重为：

$$K = (0.8, 0.2)$$

两条 TD values 为：

$$\delta = (-2, 1)$$

则 signed write-back 是：

$$0.8(-2) + 0.2(1) = -1.4$$

输出仍然是负数。正权重没有把 -2 变为 +2，也没有取绝对值。

最容易误解

“delta 必须在 attention 外面”容易引起歧义。准确说法是：delta 不应被塞进 softmax 概率并依赖其正权重表达符号；它应作为 value coordinate，在 softmax 归一化后参与 weight-value aggregation。

这里与 Xie 的 policy evaluation 机制一致。本文新增的不是恢复符号，而是 Stage I 的动作 greedification 和 state-action tokenisation。

12. kernel leakage 从哪里来

主理论使用 one-hot state-action feature。下面的闭式公式还要求 full coverage: 恰好每个 state-action pair 对应一个 context token, 没有重复或缺失。设这些互异 token 的总数为 N :

正确匹配时:

$$\langle \psi(x_k), \psi(x) \rangle = 1, \quad x_k = x$$

错误匹配时:

$$\langle \psi(x_k), \psi(x) \rangle = 0, \quad x_k \neq x$$

对正确 token, softmax 质量为:

$$K_\eta(x, x) = \frac{e^\eta}{e^\eta + N - 1}$$

每个错误 token 的质量为:

$$K_\eta(x_k, x) = \frac{1}{e^\eta + N - 1}$$

相对 identity column 的 L1 deviation 为:

$$\varepsilon_K = 2(1 - K_\eta(x, x))$$

$$\varepsilon_K = \frac{2(N - 1)}{e^\eta + N - 1}$$

因此 eta 越大, 写回越接近正确的 state-action pair。

13. leakage 的正确解释

若 TD value 满足：

$$|\delta_k^\beta| \leq B$$

真实 kernel K 与理想 identity kernel I 之间造成的一步差异满足：

$$\begin{aligned} & \left| \sum_k (K(k, x) - I(k, x)) \delta_k^\beta \right| \\ & \leq B \sum_k |K(k, x) - I(k, x)| \end{aligned}$$

所以 kernel 的一步更新误差不超过：

$$\alpha B \epsilon_K$$

这个证明使用的是 triangle inequality 和 TD bound。

最容易误解

这是一项 one-step / finite-step distortion bound。确定性 full-coverage 情况下，非 identity 但可逆的 kernel 可能只改变收敛速度而不改变 fixed point。因此不能把 ϵ_K 一概称为不可消除的渐近误差地板。

14. 参数化路线为什么出现 $\delta_t \phi_t$

考虑线性 action value:

$$Q_w(s, a) = \phi(s, a)^\top w$$

把 TD target 暂时看成 stop-gradient 常量 y_k , 并定义 squared TD loss:

$$L(w) = \frac{1}{2n} \sum_k (y_k - \phi_k^\top w)^2$$

令:

$$\delta_k = y_k - \phi_k^\top w$$

则梯度为:

$$\nabla_w L(w) = -\frac{1}{n} \sum_k \delta_k \phi_k$$

梯度下降更新因此是:

$$w^+ = w + \frac{\alpha}{n} \sum_k \delta_k \phi_k$$

所以 $\delta_t \phi_t$ 不是人为添加的技巧, 而是对参数 w 求 semi-gradient 后必然出现的样本级动态乘积。

15. 为什么我们选择绕开 $\delta \times \phi$

如果一个 token 同时包含动态字段 δ 和 ϕ ，固定线性 value projection 只能产生：

$$Vz = A\delta + B\phi + c$$

它不能仅靠线性投影产生任意 bilinear cross-term $\delta \times \phi$ 。linear attention 中类似 $H \times H$ -transpose 的结构提供样本字段之间的二阶交叉量，因此更适合选取并组合出 $\sum \delta \times \phi$ 这类项。

但这不是对所有深度、所有 head、所有 tokenisation 的 universal impossibility theorem。

本文的结构选择

本文改变学习对象：不再维护全局 w ，而是直接维护每个 query 的 $Q(x)$ ：

$$\Delta Q(x) = \alpha \sum_k K(x_k, x) \delta_k$$

ϕ 或 ψ 只负责计算地址匹配 K ， δ 作为标量 value 被聚合，因此不再需要生成向量乘积 $\delta \times \phi$ 。

答辩表述

最准确的词是“绕开”：我们通过 tabular Q-token memory 取消了参数更新中的动态乘积，而不是证明标准 softmax 已经实现了任意 $\delta \times \phi$ 。

16. 两阶段完整数据流

一次更新依次执行：

- 1 读取每个 s'_k 下的动作值 tokens;
- 2 Stage I 在同一状态的动作组内计算 M_beta ;
- 3 线性组合 reward、 M_beta 和当前 Q , 形成 signed delta;
- 4 Stage II 根据 state-action feature 计算 kernel;
- 5 以 delta 为 value, 写回所有 Q queries。

公式链为：

$$\{Q(s'_k, b)\}_{b \in \mathcal{A}} \rightarrow M_\beta Q(s'_k)$$

$$M_\beta Q(s'_k) \rightarrow \delta_k^\beta$$

$$(K_\eta, \delta^\beta) \rightarrow Q_{l+1}$$

对应总入口：**model.py::TwoStageSoftmaxQControl**。

为什么这样做

两个 softmax 不能混为一个：第一个沿 action 维归一化，第二个沿 context transition 维归一化；它们的 query、key、value 和任务都不同。

17. 理想误差递推：先做代数分解

先考虑 full coverage、identity write-back、没有 sampling error 的理想更新，并假设：

$$0 < \alpha \leq 1, \quad 0 \leq \gamma < 1, \quad \beta > 0$$

更新为：

$$Q_{l+1} = (1 - \alpha)Q_l + \alpha T_\beta Q_l$$

最优 Q 满足：

$$Q^* = T^* Q^*$$

相减，并在右侧加入一个随后又被减去的相同中间项：

$$\begin{aligned} Q_{l+1} - Q^* &= (1 - \alpha)(Q_l - Q^*) \\ &\quad + \alpha(T_\beta Q_l - T^* Q_l) \\ &\quad + \alpha(T^* Q_l - T^* Q^*) \end{aligned}$$

三项依次是：保留的旧误差、softmax-max perturbation、Bellman optimality 传播后的误差。

18. 理想误差递推：再取无穷范数

令：

$$e_\ell = \|Q_\ell - Q^*\|_\infty$$

对上一页三项使用 triangle inequality。第一项是保留的旧误差；第二项使用 Stage I 的 uniform bound；第三项使用 T-star contraction：

$$\|T^* Q_\ell - T^* Q^*\|_\infty \leq \gamma e_\ell$$

于是：

$$e_{\ell+1} \leq [(1 - \alpha) + \alpha\gamma]e_\ell + \alpha\gamma \frac{\log|\mathcal{A}|}{\beta}$$

定义：

$$\rho = 1 - \alpha(1 - \gamma) < 1$$

得到核心递推：

$$e_{\ell+1} \leq \rho e_\ell + \alpha\gamma \frac{\log|\mathcal{A}|}{\beta}$$

为什么这样做

证明借用的是 T-star contraction；不需要额外假设 M_β 或 T_β 自己是 non-expansive。

19. 为什么定理一般只给出邻域上界

把常数扰动记作：

$$c = \alpha \gamma \frac{|\log|\mathcal{A}||}{\beta}$$

反复代入递推：

$$e_\ell \leq \rho^\ell e_0 + c \sum_{j=0}^{\ell-1} \rho^j$$

利用 geometric series：

$$\sum_{j=0}^{\ell-1} \rho^j = \frac{1 - \rho^\ell}{1 - \rho}$$

又因为：

$$1 - \rho = \alpha(1 - \gamma)$$

所以：

$$e_\ell \leq \rho^\ell e_0 + \frac{\gamma |\log|\mathcal{A}||}{\beta(1 - \gamma)} (1 - \rho^\ell)$$

最终：

$$\limsup_{\ell \rightarrow \infty} e_\ell \leq \frac{\gamma |\log|\mathcal{A}||}{\beta(1 - \gamma)}$$

这个上界不等于误差地板

固定 β 时，上式给出一个不随层数消失的统一上界；它不证明每个具体问题都必然停在非零距离。这个结论是小于等于关系，不是等式，也不是非零误差的下界。

例如，若某个动作组的值全部相等：

$$q = (c, \dots, c)$$

那么对任意有限 β 都有：

$$M_\beta(q) = c = \max_a q_a$$

此时实际 softmax-max gap 为零。统一上界描述的是最坏情况下可保证的范围，不是每个问题必然达到的误差。

在固定 $0 < \alpha \leq 1$ 、 $0 \leq \gamma < 1$ 时， ρ 严格小于 1。若逐层锐度 β_l 趋于无穷，则 forcing term c_l 趋于零；稳定递推 $e_{l+1} \leq \rho e_l + c_l$ 随之推出 e_l 趋于零。

20. 加入 kernel 与 finite-context error

假设在所分析的每一层和每一个 query 上, signed TD value 都满足绝对值不超过 B 。令 $T\text{-hat-beta}$ 表示 empirical Bellman operator, 并假设:

$$\|\hat{T}_\beta Q - T_\beta Q\|_\infty \leq \varepsilon_n$$

这里 B 与 ε_n 是逐层、逐 query 的统一假设, 而不是只对某一个样本成立。attention 的每个 column 权重和为 1, 所以 kernel aggregation 不会放大这个 sup-norm target error。再加入 kernel distortion, 得到 one-step decomposition:

$$\begin{aligned} e_{l+1} &\leq \rho e_l \\ &\quad + \alpha \gamma \frac{\log |\mathcal{A}|}{\beta} \\ &\quad + \alpha B \varepsilon_K + \alpha \varepsilon_n \end{aligned}$$

三项误差需要不同处理:

- action-softmax error: 增大或调度 β ;
- kernel distortion: 增大 η 或改善 state-action representation;

- finite-context error: 增加 transition 覆盖
并做 concentration analysis。

这一步还没有给出样本复杂度

当前结果没有进一步推导 ϵ_n 随样本数变化的 sample-complexity rate。要得到这类结论，还需要规定 reward bound、transition 的独立性或依赖结构、state-action coverage，并选择相应的 concentration inequality。

因此，这里给出的是一步误差分解：它说明有限上下文误差进入哪个位置，但没有回答需要多少条 transition 才能以给定概率达到某个 ϵ_n 。该问题属于后续工作。

最容易误解

增大 β 不能解决 kernel leakage；增大 η 也不能解决 finite-sample error。两个 softmax sharpness 参数负责不同维度。

21. Q 误差怎样变成策略性能保证

假设：

$$\|Q - Q^*\|_{\infty} \leq \varepsilon$$

令策略 π_Q 对近似 Q 贪心，动作 a^* 对最优 Q 贪心。由近似 Q 的贪心性：

$$Q(s, \pi_Q(s)) \geq Q(s, a^*)$$

再在两端分别使用统一的 epsilon approximation：

$$Q^*(s, a^*) \leq Q(s, a^*) + \varepsilon$$

$$Q(s, \pi_Q(s)) \leq Q^*(s, \pi_Q(s)) + \varepsilon$$

合并可得每个状态的一步 action loss：

$$V^*(s) - Q^*(s, \pi_Q(s)) \leq 2\varepsilon$$

从一步 action loss 到长期 performance

最优 Q 在执行 $\pi_Q(s)$ 后满足：

$$Q^*(s, \pi_Q(s)) = r(s, \pi_Q(s)) + \gamma \mathbb{E}[V^*(S')]$$

而策略 π_Q 的 Bellman equation 为：

$$V^{\pi_Q}(s) = r(s, \pi_Q(s)) + \gamma \mathbb{E}[V^{\pi_Q}(S')]$$

把这两个等式代入一步 action loss，沿未来递推：

$$\|V^* - V^{\pi_Q}\|_{\infty} \leq 2\varepsilon + \gamma \|V^* - V^{\pi_Q}\|_{\infty}$$

因此：

$$\|V^* - V^{\pi_Q}\|_{\infty} \leq \frac{2\varepsilon}{1-\gamma}$$

这是从 action-value approximation 到 policy performance 的最后一座桥。

最容易误解

这是最终 greedy policy 的性能界，不证明每一个有限、随机更新层都产生单调策略改进。

22. 我们解决了什么

- 1 纠正了 “softmax 正权重导致 TD 符号消失” 的错误诊断。
- 2 用 grouped softmax 显式近似 Bellman optimality 中的 max。
- 3 用 Xie 型 kernel 完成 signed state-action TD write-back。
- 4 通过 Q-token memory 绕开全局参数更新中的 delta times phi。
- 5 给出 finite-temperature、kernel 与 finite-context 三类误差的分解。
- 6 把 Q approximation bound 转化为 greedy-policy performance bound。
- 7 主路径与 exact-max oracle 在代码中严格分离。

实验支持理论趋势：在 20 个随机 MDP 上，beta 从 1 增至 20 时，平均 Q infinity error 从约 0.2465 降至约 0.0113；观测 max gap 始终低于 entropy bound。

23. 我们尚未解决什么

- 1 结果目前主要是 finite/tabular state-action tokenisation。
- 2 它没有直接覆盖连续状态或超大动作空间。
- 3 group mask 和每个下一状态的 action-token 组是结构假设。
- 4 固定有限 beta 不能对所有动作值向量都精确实现 max。
- 5 full-coverage 理论之外，还需要更细的 sample-complexity analysis。
- 6 这是 fixed algebraic parameters 的 operator-level construction，不证明普通 pretrained Transformer 必然自动学出该算法。
- 7 我们没有证明任意多层、多头 softmax Transformer 都不能形成 delta times phi。
- 8 外部环境执行动作时仍可使用 argmax；本文的架构声明只针对 Bellman target 与 query-value update。

答辩表述

科研贡献不要求“解决所有问题”。更重要的是准确划定：前人缺什么、本文增加什么、定理在什么条件下成立、还有哪些开放问题。

24. 答辩速记

一句话贡献

我们沿用 Xie 型 signed kernel TD mechanism, 给出一个 operator-level 的 tokenised Q-control extension: Stage I 用 grouped softmax 近似 greedification, Stage II 用 state-action kernel 写回 signed TD values。

一句话难点

真正困难的是 Bellman optimality 的动作比较和参数化路线中的动态 delta times ϕ , 而不是 TD error 的正负号。

一句话理论

有限温度的 max gap 由 $\log|A| / \beta$ 统一控制; 结合 T-star contraction 可得到 Q-star 邻域保证, 并进一步转化为 greedy-policy performance bound。

一句话边界

我们绕开而非普遍解决 delta times ϕ ; 当前结论是 finite/tabular、operator-level、approximate-control 构造。